

1. External Calls Transfer Control

A Solidity contract does not execute in isolation. When it calls an untrusted address, control moves to that address until the external call returns. The recipient can execute arbitrary code, call other contracts, or call back into the original contract.

Reentrancy happens when the original contract can be entered again before its first invocation has finished restoring all invariants. The second invocation observes an intermediate state that the developer never intended to expose.

The important boundary is control flow rather than Ether alone. Any external interaction can transfer control, including token hooks, callback interfaces, fallback functions, and calls to contracts that may later be upgraded.

A useful review question is: which state assumptions must remain true if the next external call immediately invokes this function again?

2. The Vulnerable Withdrawal Pattern

Consider a vault that reads a user's balance, sends Ether to the user, and only then sets the stored balance to zero. The send is an interaction, while clearing the balance is an effect that occurs too late.

During the Ether transfer, the receiver's fallback function runs. Because the vault still records the old balance, the fallback can call withdraw again and pass the same balance check a second time.

Each nested call sends funds using the same stale accounting value. The call stack eventually unwinds, but setting the balance to zero after every send cannot recover Ether that has already left the vault.

The vulnerable ordering is check, interaction, effect. It leaves a temporary but externally observable violation of the accounting invariant.

3. Building and Tracing an Exploit

An attacker contract first deposits enough Ether to receive a legitimate vault balance. It then starts one withdrawal and uses its receive function to request another withdrawal whenever the vault sends Ether back.

The exploit needs a stopping condition. A realistic receive function checks the vault balance or a recursion counter before calling back, because unbounded recursion will run out of gas and revert the entire transaction.

Trace the exploit as a sequence: withdraw number one reads the balance, transfer number one enters the attacker, withdraw number two reads the same balance, and transfer number two enters the attacker again.

Tests should assert both storage and asset conservation. Observing only the attacker's final balance can hide accounting corruption or a revert that invalidated the apparent trace.

4. Checks-Effects-Interactions

Checks-Effects-Interactions changes the order: validate preconditions, commit all internal state changes, and perform external interactions last. A repeated call then observes the updated state and fails the original check.

For the vault, the contract copies the withdrawal amount, sets the user's balance to zero, and only then transfers Ether. If the receiver calls back, the nested invocation sees a zero balance.

The pattern protects an invariant only when every related state variable is updated before control leaves the contract. Updating one balance while leaving a shared total stale can preserve a cross-function attack path.

A failed external call must be handled deliberately. Reverting restores the earlier state atomically; silently ignoring failure can leave the user debited without receiving funds.

5. Reentrancy Guards

A reentrancy guard records whether a protected execution is already in progress. A nested call to another function using the same guard reverts before executing its body.

OpenZeppelin's `ReentrancyGuard` exposes the `nonReentrant` modifier. Protected external entry points should delegate shared work to private or internal functions because two `nonReentrant` functions cannot call one another directly.

A guard is defense in depth, not a substitute for sound state ordering. It may miss an unguarded entry point, and separate locks can fail to protect invariants shared across functions.

Guard placement should follow the invariant boundary. Apply the same lock to every external path that reads or writes the state that must change atomically.

6. Cross-Function and Read-Only Reentrancy

Cross-function reentrancy occurs when a callback enters a different function that depends on state left inconsistent by the first function. Protecting only the original function is insufficient when both functions share an invariant.

Cross-contract reentrancy extends the same problem across multiple contracts. A callback may enter a sibling contract that reads the first contract's temporary state and makes a decision from an invalid price or balance.

Read-only reentrancy does not need to mutate the vulnerable contract. Another protocol can call a view function during the inconsistent window and consume a price, exchange rate, or balance that should never have been observable.

Reviewers should map invariants and external call sites across the system, rather than searching only for a function that calls itself.

7. Pull Payments and Safer Transfers

Pull payments separate accounting from delivery. The protocol records an amount owed, and the recipient later withdraws it through a small, carefully protected function.

Using `call` is generally preferred for Ether transfer because fixed gas stipends are brittle as opcode costs evolve. Success must be checked, and the surrounding function must still assume the recipient is adversarial.

Changing transfer to `call` does not create reentrancy by itself. The vulnerability exists when an external call exposes inconsistent state; forwarding gas merely makes callback behavior easier to exercise.

For tokens, engineers must consider hooks and unusual token implementations. A familiar interface does not guarantee that an interaction is free of callbacks.

8. Testing and Review Checklist

Write an attacker contract in the test suite and exercise the complete callback path. Confirm that the protected transaction reverts or that a nested call observes already-updated accounting.

Use invariant tests to assert that total liabilities never exceed controlled assets and that a user's withdrawal cannot reduce another user's claim. Vary call order, recipients, and token behavior.

List every external call, including callbacks hidden behind libraries and token standards. For each call, identify the state that is temporarily inconsistent and every public function that can observe it.

A complete defense combines correct state ordering, appropriately scoped locks, explicit failure handling, adversarial tests, and monitoring of assumptions introduced by future upgrades.